In [ ]:
```
#23BDS1155 Divyanshu Sharma
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.metrics import silhouette_score, adjusted_rand_score
```

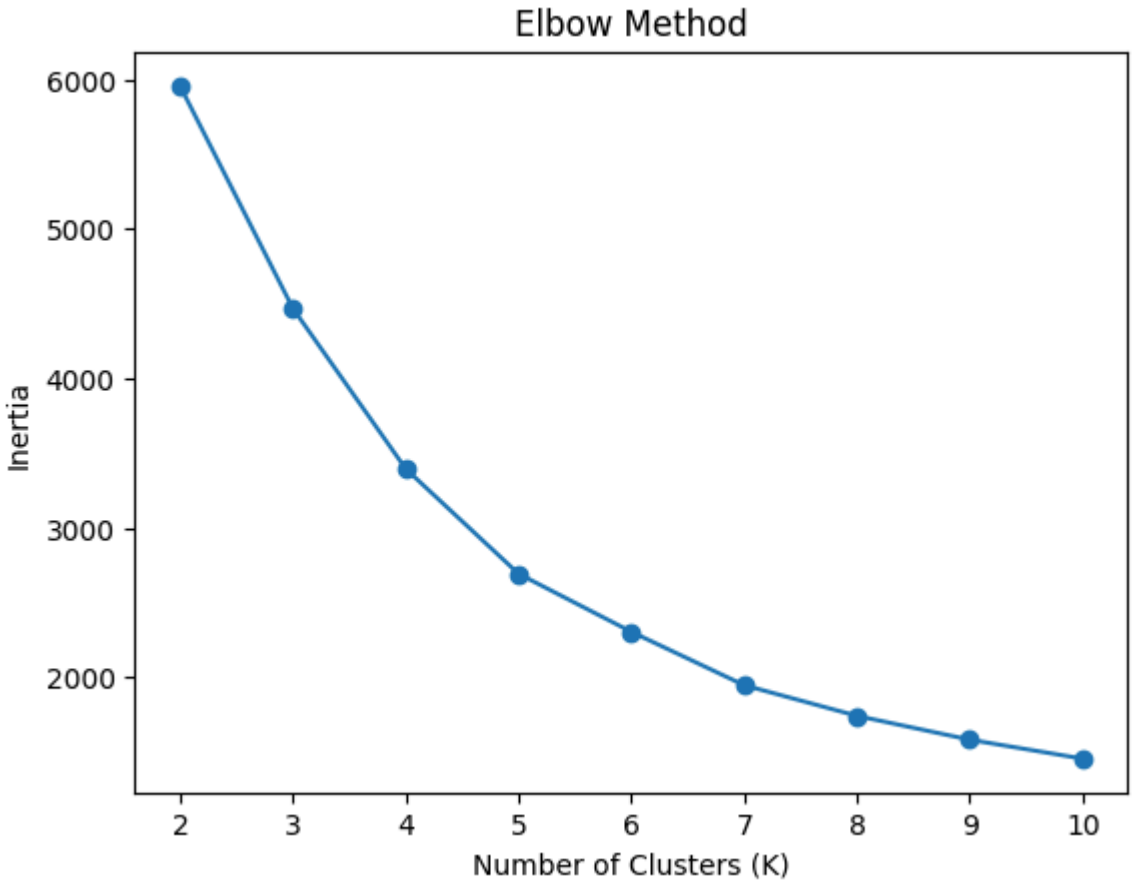
In [10]:
```
def load_and_prepare_data(path):
    df=pd.read_csv(path)

    selected_features = [
        'Monthly Charge',
        'Tenure in Months',
        'Total Charges',
        'Total Long Distance Charges'
    ]

    X=df[selected_features]
    X=X.fillna(X.mean())
    return df, X
```

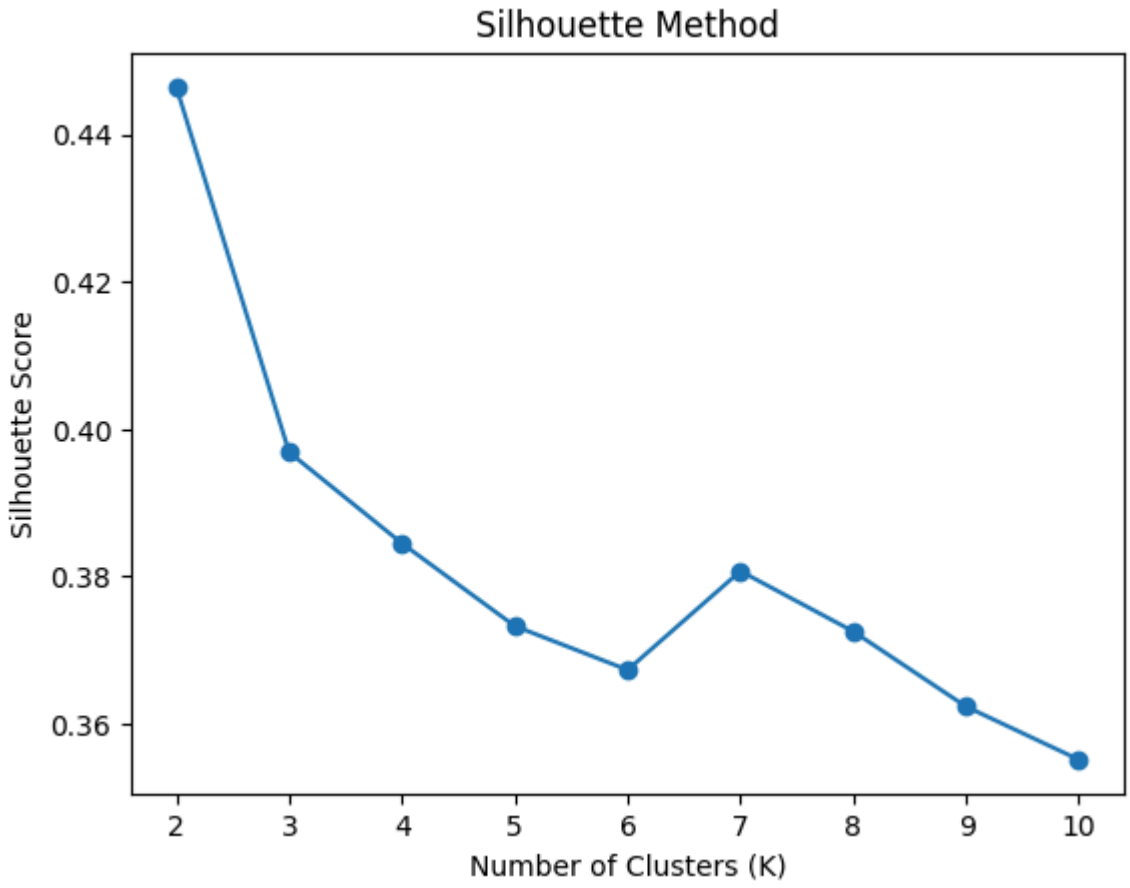
In [11]:
```
def scale_and_split(X):
    X_train, X_test = train_test_split(X, test_size=0.3, random_state=42)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    return X_train, X_train_scaled, X_test_scaled
```

In [12]:
```
def find_optimal_k(X_train_scaled):
    silhouette_scores = []
    inertia_values = []
    k_range = range(2, 11)
    for k in k_range:
        model=KMeans(n_clusters=k, random_state=42, n_init=10)
        labels=model.fit_predict(X_train_scaled)

        inertia_values.append(model.inertia_)
        silhouette_scores.append(silhouette_score(X_train_scaled, labels))
    plt.figure()
    plt.plot(k_range, inertia_values, marker='o')
    plt.xlabel("Number of Clusters (K)")
    plt.ylabel("Inertia")
    plt.title("Elbow Method")
    plt.show()
    plt.figure()
    plt.plot(k_range, silhouette_scores, marker='o')
    plt.xlabel("Number of Clusters (K)")
    plt.ylabel("Silhouette Score")
    plt.title("Silhouette Method")
    plt.show()
    best_k = k_range[np.argmax(silhouette_scores)]
    print("Optimal K (Silhouette):", best_k)
    return best_k
```

In [13]:
```
def evaluate_kmeans(df, X_train, X_train_scaled, optimal_k):
    model = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
    train_labels = model.fit_predict(X_train_scaled)
    true_labels = df.loc[X_train.index, 'Churn']
    ari = adjusted_rand_score(true_labels, train_labels)
    print("Adjusted Rand Index:", ari)
    plt.figure()
    plt.scatter(X_train_scaled[:, 0],
                X_train_scaled[:, 1],
                c=train_labels)

    plt.xlabel("Monthly Charge (Scaled)")
    plt.ylabel("Tenure in Months (Scaled)")
    plt.title("KMeans Clustering Result")
    plt.show()
```

In [14]:
```
df, X = load_and_prepare_data("train-telecom-churn.csv")
X_train, X_train_scaled, X_test_scaled = scale_and_split(X)
optimal_k = find_optimal_k(X_train_scaled)
evaluate_kmeans(df, X_train, X_train_scaled, optimal_k)
```






```
Optimal K (Silhouette): 2
Adjusted Rand Index: -0.012779784996842913
```

In [15]:
```
# Q2
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
from sklearn.preprocessing import OrdinalEncoder
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
```

In [16]:
```
def load_and_clean_adult(path):
    adult=pd.read_csv(path)
    categorical_features = [
        'workclass', 'education', 'marital.status',
        'occupation', 'relationship',
        'race', 'native.country'
    ]
    X = adult[categorical_features]
    X = X.replace(' ?', np.nan)
    X = X.fillna("Unknown")
    encoder = OrdinalEncoder()
    X_encoded = encoder.fit_transform(X)
    return X_encoded
```

In [17]:
```
def agglomerative_analysis(X_encoded):
    best_score = -1
    best_k = 0
    for k in range(2, 6):
        model = AgglomerativeClustering(
            n_clusters=k,
            metric='hamming',
            linkage='average'
        )
        labels = model.fit_predict(X_encoded)
        score = silhouette_score(X_encoded, labels, metric='hamming')
        print(f"K ={k}, Silhouette Score = {score}")
        if score > best_score:
            best_score = score
            best_k=k
    print("\nBest K:", best_k)
    final_model = AgglomerativeClustering(
        n_clusters=best_k,
        metric='hamming',
        linkage='average'
    )
    final_labels = final_model.fit_predict(X_encoded)
    print("\nCluster Distribution:")
    print(pd.Series(final_labels).value_counts())
    plt.figure()
    plt.scatter(X_encoded[:, 0],
                X_encoded[:, 1],
                c=final_labels)
    plt.xlabel("workclass (encoded)")
    plt.ylabel("education (encoded)")
    plt.title("Agglomerative Clustering Result")
    plt.show()
```

In [18]:
```
X_encoded = load_and_clean_adult("adult.csv")
agglomerative_analysis(X_encoded)
```

```
K =2, Silhouette Score = 0.33642393380156327
K =3, Silhouette Score = 0.2560971240402595
K =4, Silhouette Score = 0.21473737818957955
K =5, Silhouette Score = 0.15923022457296296

Best K: 2

Cluster Distribution:
0    32527
1       34
Name: count, dtype: int64
```